

QEMU

In the first article of this series, we covered the basic use of QEMU. Now let's dig deeper into its abilities, looking at the embedded domain.

Part—2

Techies who work in the embedded domain must be familiar with the ARM (Advanced RISC machine) architecture. In the modern era, our lives have been taken over by mobile devices like phones, PDAs, MP3 players and GPS devices that use this architecture. ARM has cemented its place in the embedded devices market because of its low cost, lower power requirements, less heat dissipation and good performance. Purchasing ARM development hardware can be an expensive proposition—many thanks to the QEMU developers who added the functionality of emulating the ARM processor to QEMU.

You can use QEMU for two purposes in this arena—to run an ARM program, and to boot and run the ARM kernel. In the first case, you can run and test ARM programs without installing ARM OS or its kernel. This feature is very helpful and time-saving. In the second case, you can try to boot the Linux kernel for ARM, and test it.

Compiling QEMU for ARM

In the last article, we compiled QEMU for x86. This time let's compile it for ARM. Download the QEMU source, if you don't have it, from <http://download.savannah.gnu.org/releases/qemu/qemu-0.14.0.tar.gz>. Extract the tar ball, change to the extracted directory, configure and build it as follows:

```
# tar -zxvf qemu-0.14.0.tar.gz
# cd qemu-0.14.0
# ./configure --target-list=arm-softmmu
# make
# make install
```

You will find two output binaries, *qemu-arm* and *qemu-system-arm*, in the source code directory. The first is used to execute ARM binary files, and the second to boot the ARM OS.

Obtaining an ARM tool-chain

Let's develop a small test program. Just as you need the

x86 tool-chain to develop programs for Intel, you need the ARM tool-chain for ARM program development. You can download it from <http://www.codesourcery.com/sgpp/lite/arm/portal/package7813/public/arm-none-eabi/arm-2010.09-50-arm-none-eabi-i686-pc-linux-gnu.tar.bz2>.

Extract the archive's contents, and view a list of the available binaries:

```
# tar -jxvf arm-2010.09-50-arm-none-linux-gnueabi-i686-pc-linux-
gnu.tar.bz2
# cd arm-2010.09/bin/
# ls
-rwxr-xr-x 1 root root 569820 Nov 7 22:23 arm-none-linux-
gnueabi-addr2line
-rwxr-xr-x 2 root root 593236 Nov 7 22:23 arm-none-linux-
gnueabi-ar
-rwxr-xr-x 2 root root 1046336 Nov 7 22:23 arm-none-linux-
gnueabi-as
-rwxr-xr-x 2 root root 225860 Nov 7 22:23 arm-none-linux-
gnueabi-c++
-rwxr-xr-x 1 root root 572028 Nov 7 22:23 arm-none-linux-
gnueabi-c++filt
-rwxr-xr-x 1 root root 224196 Nov 7 22:23 arm-none-linux-
gnueabi-cpp
-rwxr-xr-x 1 root root 18612 Nov 7 22:23 arm-none-linux-
gnueabi-elfedit
-rwxr-xr-x 2 root root 225860 Nov 7 22:23 arm-none-linux-
gnueabi-g++
-rwxr-xr-x 2 root root 222948 Nov 7 22:23 arm-none-linux-
gnueabi-gcc
```

Cross-compiling and running the test program for ARM

Now use the *arm-none-linux-gnueabi-gcc* tool to compile a test C program. Before proceeding, you should add the ARM tool-chain to your PATH:

```
# PATH=$(Your-path)/arm-2010.09/bin:$PATH
```